

Entra ID web-app sign-in, extended with Zscaler proxy authentication

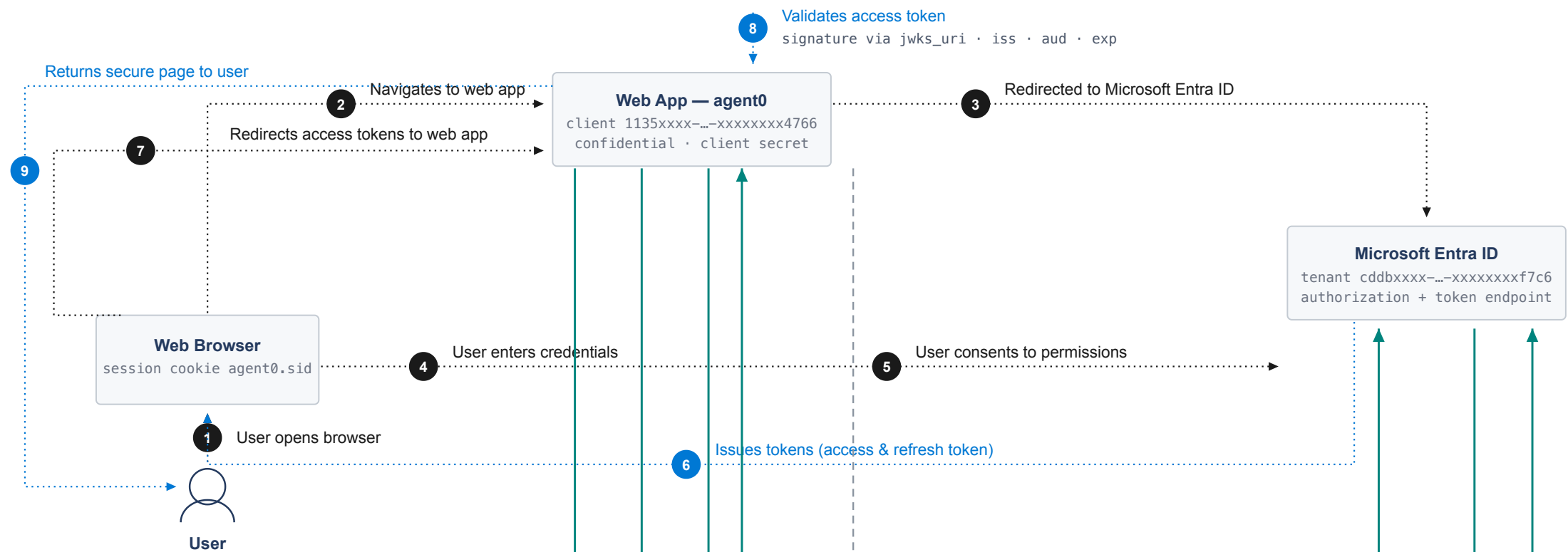
Steps 1–9 are Microsoft's own OpenID Connect web-app diagram, unchanged. Steps 10–17 are this reference implementation's extension: how the agent reaches the internet through an authenticated Zscaler explicit proxy *without a second sign-in*, by reusing the token the web-app flow already produced.

Henry Shen · Zscaler

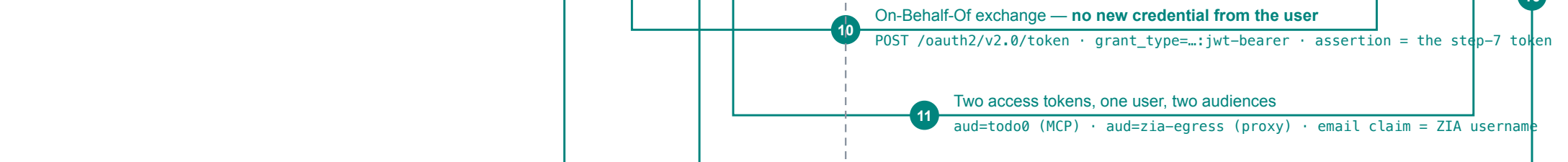
Base diagram: learn.microsoft.com/en-us/entra/architecture/auth-oidc · Extension built and verified on agent0 / todo0. Identifiers shown are masked values from the reference lab.

Personal work by Henry Shen — not an official Zscaler solution. It does not represent Zscaler's official position and is not supported by Zscaler; it shares one way to configure and consume Zscaler products, **for educational purposes only — not for production use.**

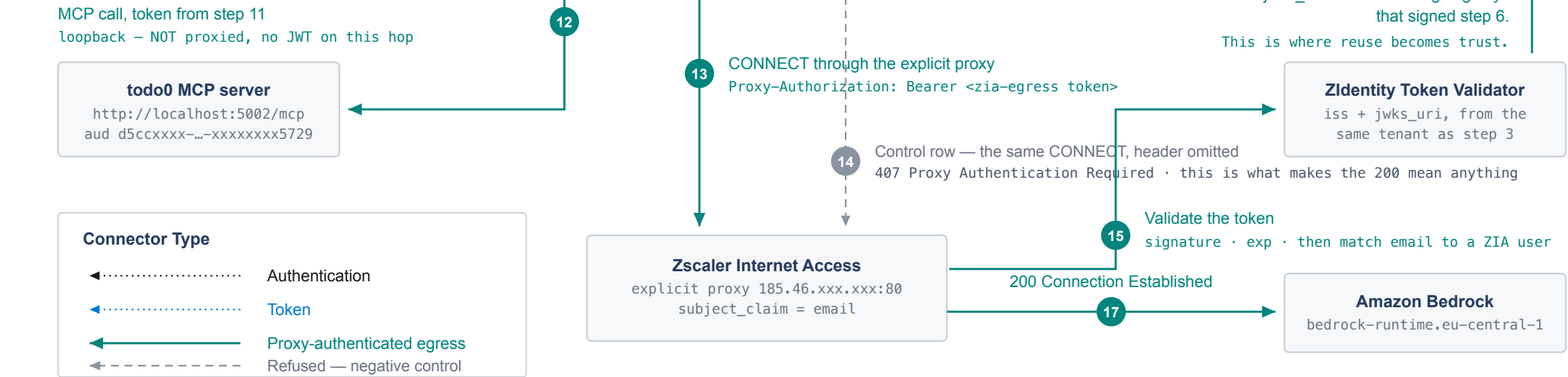
1-9 · WEB APP SIGN-IN — MICROSOFT'S FLOW, UNCHANGED



10-11 · TOKEN REUSE — ON-BEHALF-OF, NO SECOND SIGN-IN



12-17 · EGRESS — THE SAME IDENTITY, AUTHENTICATED AT THE PROXY



Steps 1-9 and the Authentication / Token connector styles are Microsoft's. Steps 10-17, the third and fourth connector types, and the four boxes drawn with a teal outline are this reference implementation's extension. The extension adds no new sign-in: every token below the first divider descends from the access token issued at step 6. Masked lab identifiers in the drawing: `client` = <AGENT0_CLIENT_ID>, `tenant` = <TENANT_ID>, `todo0 aud` = <TODO0_APP_ID>, `explicit proxy` = <ZIA_PROXY>.

What is actually reused

"Reuse" here is four specific things, not a general resemblance. Each one is why the proxy leg needs no second login and no second credential store:

1. **The sign-in itself.** Steps 1–9 happen once, in the browser. The user never sees a prompt for the proxy. Steps 10–17 are entirely server-to-server.
2. **The user's access token, as the assertion.** The On-Behalf-Of exchange at step 10 presents the *same* token the web app received at step 7 as its `assertion`. Entra therefore knows the request is made on behalf of a user who has already authenticated interactively — including any MFA and the consent from step 5.
3. **The signing keys.** The Token Validator at step 16 fetches `jwks_uri` from the same tenant that signed step 6. It is not a separate trust anchor bolted on for the proxy; the proxy trusts exactly the tokens the sign-in produced. **This is the load-bearing reuse** — it is what makes the identity on the wire the same identity that logged in.
4. **The identity, end to end.** The `oid` from the id token is the `userId` on the database row the agent writes through MCP; the `email` claim from the same token is the username Zscaler logs against the Bedrock request. One login, one human, visible in both planes.

Where this diagram deliberately differs from Microsoft's

Source diagram	What agent0 actually does	Why the difference matters
6 — "Issues tokens" to the browser; 7 — "Redirects access tokens to web app"	Authorization code + PKCE. Step 6 delivers a <code>code</code> ; step 7 hands that code to <code>/callback</code> ; agent0 exchanges it server-side with its client secret for the id token and access token.	No token ever transits the browser, and the token that becomes the OBO assertion at step 10 only ever exists server-side. The Learn diagram's shape would put it in the user agent.
"Web App" as a single box	Two registered applications — <code>agent0</code> and <code>todo0</code> — each with its own client id, its own consent grant and its own session cookie.	First-time consent (step 5) is per application . agent0 consenting does not consent todo0, even for the same user and the same three scopes.
Flow ends at step 9 with a secure page	Step 9 is where the interesting part <i>starts</i> : the session now holds a token that can be exchanged for other audiences.	The proxy leg is a consequence of the web-app flow, not a parallel mechanism beside it.

The two things this does not do

The MCP hop at step 12 is not proxy-authenticated, and cannot be. It is a loopback call inside one container. Proxy authentication only applies to *explicit-proxy egress*, so there is no JWT on that hop; the token it carries is the todo0-audience OAuth token, which is a different control. A diagram that drew a Zscaler connector on step 12 would be wrong.

The proxy must be explicit, not a default route. Traffic that reaches the internet over a plain default-route tunnel bypasses JWT authentication *silently and open* — it succeeds, and nothing in the response says the identity was never checked. The tell that authentication is genuinely in force is step 14: omit the header and the proxy must answer `407` with a Bearer challenge. If omitting it still yields a connection, the egress path is not the one drawn here.

Session level and process level are two different identities

Step 11 mints a token per *logged-in session*. An agent also makes calls that belong to no session — start-up, health checks, background work — and those need a separate, process-level identity of their own. The two are deliberately not fallbacks for each other: a session with no token must fail with a `407` rather than quietly borrow the process identity, because attribution in the proxy log is the entire point of doing this.

In this reference implementation the Entra path ships the **session level only** — `GET /api/zia/status` reports `{"session": {...}, "process": null}`. A process-level workload identity is an extension point: it needs a non-interactive credential of its own, and must stay separate from every user session.

What has to exist on the Zscaler side

A ZIdentity Token Validator for the Entra issuer. Issuer

`https://login.microsoftonline.com/<TENANT_ID>/v2.0` , JWKS URL
`.../<TENANT_ID>/discovery/v2.0/keys` , Subject Claim `email` , and a required claim `aud = <ZIA_EGRESS_APP_ID>` — the last is what stops a token issued for another application in the same tenant from authenticating to the proxy. Masked lab example:

`https://login.microsoftonline.com/cddbxxxx-xxxx-xxxx-xxxx-xxxxxxxxxf7c6/v2.0` , `aud = a015xxxx-xxxx-xxxx-xxxx-xxxxxxxxxc4d6` .

A ZIA user per identity, whose username equals the token's `email` claim — which Entra fills from the user's `mail` attribute. A token whose `email` matches no ZIA user is validly signed and still refused with `407` .

The explicit proxy, with JWT authentication enabled for that traffic — `<ZIA_PROXY>` , e.g.

`185.46.xxx.xxx:80` or `gateway.<zscaler-cloud>.net:80` . ZIdentity's "Test Token" checks only signature and expiry; the acceptance test is step 14's `407` next to step 17's `200` .

Extension verified end to end: headless sign-in through Entra's own login pages → `strategy: obo` , `connected: true` → a todo written under the user's `oid` (`userId=9b5bxxxx-...`) → read back by `todo0` under its own independent login. Base diagram © Microsoft, from the Entra architecture documentation. This document is personal work by Henry Shen, not an official Zscaler solution, and is not supported by Zscaler. Educational use only — not for production. For supported configurations, refer to Zscaler's official documentation.